

Contents

Why MFA Bypass Still Works — Even in 2026	3
MFA Fatigue Attacks (Push Bombing)	3
How It Works	3
Step-by-Step: How an Attacker Launches Push Bombing	3
Real-World Example	4
Defense Tips	4
2. Reverse Proxy Phishing (Adversary-in-the-Middle)	4
What's Going On Here?	4
Tools and Frameworks	4
Step-by-Step: Reverse Proxy Phishing in Action	4
Sample Evilginx3 Phishing Configuration	5
Pro Tip	5
3. Exploiting Legacy Authentication Protocols	5
Why This Still Works	5
Attack Walkthrough	5
Practical Scanning Example	5
What to Fix	5
4. SIM Swapping and Phone Number Hijacking	6
The Oldest Trick, Upgraded	6
Stage-by-Stage Attack	6
Defense Measures	6
5. OAuth Token Phishing and Consent Grant Attacks	6
The Modern OAuth Phish	6
The Attack Process	6
Example Consent URL	7
Why It's Effective	7

6. Man-in-the-Endpoint: Token Extraction via Malware	7
It's Not Just About Phishing	7
How Attackers Pull It Off	7
A Simple Token Stealer Example	8
What Really Happens	8
7. MFA Reset and Social Engineering Helpdesk	8
The Human Weak Spot	8
The Social Engineering Game Plan	8
Tips for Defense	8
8. Exploiting Self-Enrollment and Registration Loopholes	9
New Account, Who Dis?	9
The Exploit Path	9
An Example Exploit (Python Automation)	9
9. Exploiting SSO and Federated Identity Gaps	9
Where SSO Gets Messy	9
Typical Attack Steps	10
Real-World SAML Assertion Tampering (Simplified)	10
What's the Fix?	10
10. Pass-the-Cookie and Pass-the-Token Attacks	10
What's Old Is New Again	10
Step-by-Step: Pass-the-Cookie	10
How to Replay a Cookie	11
Pro-level Automation	11
The Catch	11
Takeaways: Defending Against 2026's MFA Bypass Arsenal	11
□ Become a VeryLazyTech Member — Get Instant Access	12
□ Need Specific Resources?	12
□ Stay in the Loop	12

You patched, you planned, you rolled out MFA — and attackers still waltz in. Sound familiar? Despite billions invested, nearly 40% of reported breaches in 2026 involved some form of MFA bypass. So, what's going wrong? Here's what the hackers aren't telling you... and how they're getting past your "unbreakable" defenses.

Why MFA Bypass Still Works — Even in 2026

Multi-factor authentication feels like a silver bullet. SMS, push notifications, biometrics — shouldn't that stop most attacks? In theory, yes. In the mess of real networks? Not so much.

Attackers love the chaos. MFA systems are everywhere, but so are misconfigurations, outdated endpoints, and users who just want to "get it over with." It's a playground for pentesters and real adversaries alike.

This guide will walk you through 10 real-world MFA bypass techniques that are being used right now — some of which I've personally tested in red team engagements, others recently surfaced in top bug bounty writeups.

Ready to peek behind the curtain? Here we go.

MFA Fatigue Attacks (Push Bombing)

How It Works

You've probably seen this trick already — it's become a go-to approach in both pentesting and real attacks. MFA fatigue leverages human impatience. Attackers flood the victim with push notifications until the user, tired or distracted, finally taps "Approve."

Step-by-Step: How an Attacker Launches Push Bombing

1. Obtain Credentials

Through phishing, credential stuffing, or other means.

1. Initiate Multiple Logins

Use the stolen credentials to repeatedly attempt logins, triggering a barrage of push notifications to the victim's phone.

1. Exploit Human Nature

Most users, after receiving 10, 20, or even 100 pop-ups, will eventually hit "Yes" just to make it stop.

Real-World Example

During a recent engagement, we tested a client's Azure AD with push-based MFA. After 40 notifications over two hours, the user — clearly frustrated — hit approve. Full access, and not a line of code needed.

Defense Tips

- Switch to number-matching push or FIDO2 tokens.
- Set alerting for high-frequency MFA prompts.
- Educate users: Never approve unexpected requests.

2. Reverse Proxy Phishing (Adversary-in-the-Middle)

What's Going On Here?

Traditional phishing? Old news. Modern attackers are standing between you and your login — proxying every request in real-time, snatching your session cookies **after** you've passed MFA.

Tools and Frameworks

- Evilginx3
- Modlishka
- Muraena

Step-by-Step: Reverse Proxy Phishing in Action

1. Clone the Login Portal

Set up a phishing site that perfectly mimics the real one.

1. Configure the Proxy

Evilginx3 intercepts all traffic, relaying it to the real service but logging credentials and session tokens.

1. User Logs In (with MFA!)

The victim enters credentials, completes the MFA prompt, and thinks they're in.

1. Steal the Session Cookie

The attacker instantly replays the session elsewhere—no more MFA required.

Sample Evilginx3 Phishing Configuration

```
proxy_hosts:  
  - {phish_sub: 'login', orig_sub: 'login', domain: 'acme.com', session_cookie: 'AUTH_TOKEN'}
```

Pro Tip

Many services only check MFA at authentication, not per session. So, if you steal the session cookie, you're in.

3. Exploiting Legacy Authentication Protocols

Why This Still Works

In 2026, plenty of organizations **still** have legacy protocols enabled (think IMAP, POP3, SMTP, or old-school SMB). These often bypass MFA enforcement by design.

Attack Walkthrough

1. Find Legacy Endpoints

Use nmap, masscan, or Shodan to locate exposed legacy services.

1. Credential Stuffing/Phishing

Try passwords you've harvested. Since these protocols often don't prompt MFA, a valid username/password combo grants access.

1. Pivot and Escalate

With mailbox or file server access, escalate privileges or launch lateral movement.

Practical Scanning Example

```
nmap -p 143,993,110,995,445,target.com
```

If you see IMAP or SMB open, you might have a path.

What to Fix

- Block legacy authentication at the firewall.

- Audit and disable unnecessary protocols.

4. SIM Swapping and Phone Number Hijacking

The Oldest Trick, Upgraded

You'd think telcos would have this nailed by now, but no. Attackers still social-engineer their way into SIM cards, especially with new automated customer support bots ("I lost my phone, please help!").

Stage-by-Stage Attack

1. Research Target

Gather victim's PII from social media, data breaches, or OSINT tools.

1. Contact the Telco

Call or chat with support, impersonating the victim.

1. Convince the Rep/AI

Request a SIM transfer to attacker-controlled device. Sometimes, this takes multiple tries or "pretexting" as a family member.

1. Intercept SMS MFA Codes

With control of the victim's number, log in and receive all 2FA codes.

Defense Measures

- Use app-based authenticators, not SMS MFA.
- Employ port-out locks where available.

5. OAuth Token Phishing and Consent Grant Attacks

The Modern OAuth Phish

With everyone living in Google Workspace, Microsoft 365, or Slack, attackers are loving OAuth consent screens. Instead of stealing passwords, they trick users into granting persistent access — no need to ever log in again.

The Attack Process

1. Build a Malicious OAuth App

Register an app requesting wide permissions (read email, contacts, files).

1. Send OAuth Consent Link

Victim clicks, logs in, and sees a legit-looking "grant access" screen.

1. Victim Approves

Now the attacker has a refresh token—bypassing MFA forever, until consent is revoked.

Example Consent URL

```
https://accounts.google.com/o/oauth2/auth?client_id=attacker-app-id&redirect_uri=https://malicious.com&scope=email+profile+https://mail.google.com/&response_type=code
```

Why It's Effective

- No password phishing required.
- MFA doesn't protect the OAuth approval flow.
- Persistent, sometimes undetected access.

6. Man-in-the-Endpoint: Token Extraction via Malware

It's Not Just About Phishing

Some attackers skip human trickery and go straight for the endpoint. Custom malware or even off-the-shelf info-stealers can grab cached MFA tokens from browsers, password managers, or memory.

How Attackers Pull It Off

1. Initial Access

Classic phishing, drive-by-download, or USB drop delivers malware.

1. Token Harvesting

Malware scans disk, memory, and browser storage for session cookies, JWTs, and access tokens.

1. Replay

Extracted tokens are replayed from a different machine or VM, invalidating MFA.

A Simple Token Stealer Example

```
import os

token_path = os.path.expanduser('~/.config/google-chrome/Default/Cookies')
with open(token_path, 'rb') as file:
    data = file.read()
    # parse for auth tokens
```

(Yes, it's rough, but you get the idea.)

What Really Happens

In practice, red teams often use open source stealers tweaked to evade EDR, or write custom scripts targeting Electron apps like Slack or Teams.

7. MFA Reset and Social Engineering Helpdesk

The Human Weak Spot

Even the best MFA can be undone by a helpful IT admin. Attackers call in, claim they lost their device, and — if the support desk isn't strict — get reset instructions sent to their own account.

The Social Engineering Game Plan

1. Research Helpdesk Procedures

Figure out what information is needed to verify identity.

1. Prepare Pretext

Sometimes attackers pose as VIPs or use a sense of urgency ("CEO can't log in!").

1. Make the Call

Use spoofed numbers or VoIP to appear legitimate.

1. Convince the Agent

Obtain reset links or have MFA removed/changed.

Tips for Defense

- Require strong identity checks for resets.
- Use out-of-band verification (video, callback).

- Train helpdesk staff on common pretexts.

8. Exploiting Self-Enrollment and Registration Loopholes

New Account, Who Dis?

Many organizations allow users to enroll new devices or phone numbers with minimal checks — especially after a "lost device" scenario. Attackers abuse this to register their own MFA factors.

The Exploit Path

1. Compromise Account or Trick HR/IT

Use phishing or social engineering to access the self-enrollment portal.

1. Register a New MFA Factor

Add attacker-controlled phone or authenticator.

1. Use the New Factor for Access

Now MFA is working **for** the attacker, not against.

An Example Exploit (Python Automation)

```
import requests

url = 'https://portal.company.com/enroll'
data = {'user': 'victim', 'phone': '+15551234567'}
requests.post(url, data=data)
```

This is oversimplified, but the real thing is just a few HTTP requests away if registration isn't locked down.

9. Exploiting SSO and Federated Identity Gaps

Where SSO Gets Messy

Everybody loves SSO (Single Sign-On) for convenience. But federated identity — think SAML, OIDC — can introduce dangerous gaps if MFA policies aren't enforced **everywhere**.

Typical Attack Steps

1. Find a Weak Federated App

Look for a SaaS app or legacy service federated to the main identity provider (IdP) but not enforcing MFA.

1. Authenticate via the Weak Point

Use valid credentials to log into the federated app, which trusts the IdP's assertion — sometimes skipping MFA entirely.

1. Pivot to Internal Resources

If the federated app allows further access or has API tokens, you can escalate inside the network.

Real-World SAML Assertion Tampering (Simplified)

```
<saml:Attribute Name="MFAStatus">
  <saml:AttributeValue>not_performed</saml:AttributeValue>
</saml:Attribute>
```

If the app doesn't **require** MFAStatus = "performed", you're in.

What's the Fix?

- Enforce MFA at the IdP, not the app.
- Audit all federated integrations for policy gaps.

10. Pass-the-Cookie and Pass-the-Token Attacks

What's Old Is New Again

Just like classic pass-the-hash in Windows land, attackers now replay valid cookies or tokens snatched from endpoints or via MiTM. MFA doesn't re-prompt as long as the cookie is valid.

Step-by-Step: Pass-the-Cookie

1. Steal a Session Cookie

Extract from browser (see attack #6) or via MiTM tools.

1. Replay in Browser

Import the cookie into a fresh browser profile using dev tools or extensions.

1. Instant Access

The site trusts the valid session—no MFA prompted.

How to Replay a Cookie

1. Open Chrome DevTools (F12).
2. Go to Application → Cookies.
3. Paste the stolen cookie value for the domain.
4. Refresh the page.

Pro-level Automation

```
from selenium import webdriver

driver = webdriver.Chrome()
driver.get('https://target.com')
driver.add_cookie({'name': 'sessionid', 'value': 'STOLEN_VALUE'})
driver.refresh()
```

The Catch

Some sites now bind cookies to device fingerprints or IP addresses, but plenty still don't. Red teams use this all the time (and so do the bad guys).

Takeaways: Defending Against 2026's MFA Bypass Arsenal

You might think MFA is enough. In practice, what really happens is that attackers adapt — faster than most organizations can patch. MFA's only as strong as its weakest link: stale endpoints, tired users, or that one lazy integration.

Here's where it gets interesting — defense isn't about adding more factors, but about closing the gaps:

- Regularly audit all authentication flows, including legacy and federated paths.
- Enforce device health and context-aware logins (location, device fingerprint).
- Train users **and** your helpdesk—attackers exploit humans, not just tech.
- Rotate and invalidate tokens and sessions aggressively.
- Move to phishing-resistant factors (FIDO2, passkeys, number-matching).

And if you're testing your own defenses: try walking through these 10 techniques on your next pentest. You'll be surprised how many still work — even in 2026.

Stay curious, stay lazy (in the right way), and keep breaking stuff — before someone else does.

☐ **Become a VeryLazyTech Member — Get Instant Access**

What you get today:

☐ 70GB Google Drive **packed with cybersecurity content**

☐ 3 full courses **to level up fast**

☐ **Join the Membership** → <https://shop.verylazytech.com> (<https://shop.verylazytech.com>)

☐ **Need Specific Resources?**

☐ Instantly download the **best hacking guides, OSCP prep kits, cheat sheets, and scripts** used by real security pros.

☐ **Visit the Shop** → <https://shop.verylazytech.com> (<https://shop.verylazytech.com>)

☐ **Stay in the Loop**

Want quick tips, free tools, and sneak peeks?

✖ <https://x.com/verylazytech/> (<https://x.com/verylazytech/>)

| ☐ <https://github.com/verylazytech/> (<https://github.com/verylazytech/>)

| ☐ <https://youtube.com/@verylazytech/> (<https://youtube.com/@verylazytech/>)

| ☐ <https://t.me/+mSGyb008VL40MmVk/> (<https://t.me/+mSGyb008VL40MmVk/>)

| ☐♂ <https://www.verylazytech.com/> (<https://www.verylazytech.com/>)